

Agentic Configuration Management (ACM): A Reference Configuration Model for Governed Agentic Systems

Audrey Quessadda-Vial
PwC
audrey.quessada-vial@pwc.com

Draft — August 2026

1 Governance Semantics

The ACM reference model specifies the structural organization of governed agentic systems. This section complements the metamodel by defining the operational semantics governing lifecycle evolution, state evaluation, dependency propagation, and runtime reconstruction.

The governance semantics are intentionally defined independently from any execution framework. Their purpose is not to describe execution behavior but to provide a deterministic interpretation of governed configurations throughout their lifecycle.

Let

$$G = (V, E) \tag{1}$$

denote the ACM Configuration Graph introduced in Section ??, where each vertex represents an immutable ACI revision and each edge represents a typed governance relationship.

Each revision is associated with a governance descriptor

$$\Gamma(v) = (L(v), Q(v), A(v), I(v), El(v), Rt(v)), \tag{2}$$

where the individual components respectively denote the lifecycle, quality, assurance, impact, eligibility, and runtime states associated with revision v .

The governance descriptor is a logical representation of the governance status of an ACI revision. It does not perform any computation itself. Individual state components are evaluated independently according to the semantics introduced in the following subsections and are subsequently propagated through governance relationships according to explicitly defined propagation policies.

This separation between state spaces, evaluation functions, propagation policies, and runtime reconstruction ensures that each semantic component is defined once and reused consistently throughout the model.

1.1 Governance Lifecycle

The lifecycle dimension represents the intrinsic governance maturity of an immutable ACI revision independently of any dependency or runtime consideration. Unlike the remaining governance dimensions, lifecycle states are explicitly assigned through governance actions rather than derived by propagation.

Figure ?? presents the lifecycle state machine associated with every ACI revision.

Each revision progresses monotonically through the lifecycle. Once a revision reaches a given state, it cannot transition back to a previous lifecycle state.

This monotonic property guarantees configuration immutability while preserving complete governance traceability across successive revisions.

Lifecycle transitions are therefore constrained by a partial order

$$Draft < Validated < Approved < Released < Deprecated < Archived, \tag{3}$$

where each transition represents an explicit governance decision.

The lifecycle state is denoted

$$L(v) \in \mathcal{L}, \quad (4)$$

where $\mathcal{L}$ denotes the lifecycle state space formally defined in Appendix ??.

Unlike derived governance dimensions introduced later in this section, lifecycle states never result from dependency propagation. They constitute primary governance information from which eligibility and propagation decisions are subsequently derived.

1.2 Governance State Spaces

The governance semantics are defined over six orthogonal state spaces describing complementary aspects of an immutable ACI revision. Each state space captures an independent governance concern and is evaluated according to its own semantics.

Unlike the lifecycle state, which is explicitly assigned through governance decisions, the remaining dimensions are either derived from governance evidence, dependency analysis, or runtime observations.

Figure 1 summarizes the six orthogonal state machines used throughout the remainder of this paper.

The governance dimensions are:

- **Lifecycle** ($\mathcal{L}$): governance maturity of a revision;
- **Quality** ($\mathcal{Q}$): validation status of the revision itself;
- **Assurance** ($\mathcal{A}$): completeness of governance evidence;
- **Impact** ($\mathcal{I}$): propagation status resulting from dependency analysis;
- **Eligibility** ($\mathcal{E}$): authorization for operational use;
- **Runtime** ($\mathcal{R}$): observed execution state.

Each state space is formally defined in Appendix B. Their interaction is specified later through evaluation functions and the propagation operator.

1.3 Propagation Policies

Governance propagation is intentionally separated from state evaluation. While governance state spaces define *what* is evaluated, propagation policies define *how* governance information is allowed to traverse configuration relationships.

Each relationship type is therefore associated with a propagation policy determining the effect of governance changes on dependent revisions. Policies are intrinsic properties of relationship types and remain independent of the current governance states of the connected revisions.

A propagation policy is defined as a mapping

$$\Pi : T_R \rightarrow \mathcal{P}, \quad (5)$$

where T_R denotes the set of relationship types introduced by the reference model and $\mathcal{P}$ is the finite set of propagation policies

$$\mathcal{P} = \{Blocking, Warning, Informational, None\}. \quad (6)$$

Each policy specifies the governance significance of a relationship during propagation.

- **Blocking** indicates that governance changes affecting the source revision shall propagate to dependent revisions and may restrict their operational eligibility until reassessment is completed.
- **Warning** propagates governance information without automatically preventing operational use. The affected revision remains executable but requires governance attention.
- **Informational** records the dependency for traceability purposes without modifying governance states.
- **None** disables propagation across the relationship while preserving the structural dependency.

Propagation policies are associated with relationship types rather than individual revisions. Consequently, identical relationship types always induce identical propagation semantics independently of the connected ACI revisions.

When several propagation paths converge on the same revision, multiple propagation policies may simultaneously apply. Their combined effect is resolved deterministically according to the precedence rules defined in Appendix C. This deterministic composition guarantees that governance propagation remains independent of traversal order.

1.4 Governance Evaluation Functions

The governance state spaces introduced previously define the possible states associated with an ACI revision. Governance evaluation functions determine how these states are assigned from governance evidence while remaining independent of dependency propagation.

Three evaluation functions are defined. They respectively evaluate intrinsic quality, governance assurance, and operational eligibility. Each function produces a deterministic result for a given governance context and is evaluated independently before any propagation mechanism is applied.

1.4.1 Quality Evaluation

The quality evaluation function assigns the quality state associated with an ACI revision according to the available validation evidence.

$$f_{\text{quality}} : V \rightarrow \mathcal{Q}, \quad (7)$$

where V denotes the set of ACI revisions and $\mathcal{Q}$ is the quality state space introduced in Section 1.2.

The resulting quality state is denoted

$$Q(v) = f_{\text{quality}}(v). \quad (8)$$

Quality evaluation depends exclusively on the considered revision and is therefore independent of dependency propagation.

1.4.2 Assurance Evaluation

The assurance evaluation function determines the completeness of governance evidence associated with a revision.

$$f_{\text{assurance}} : V \rightarrow \mathcal{A}, \quad (9)$$

where $\mathcal{A}$ denotes the assurance state space.

The resulting assurance state is

$$A(v) = f_{\text{assurance}}(v). \quad (10)$$

Assurance reflects the completeness of governance evidence independently of the intrinsic quality of the revision.

1.4.3 Eligibility Evaluation

Operational eligibility depends on the combined interpretation of the governance dimensions. Unlike quality and assurance, eligibility is therefore derived from multiple governance states.

The eligibility evaluation function is defined as

$$f_{\text{elig}} : \mathcal{L} \times \mathcal{Q} \times \mathcal{A} \times \mathcal{I} \times \mathcal{P} \rightarrow \mathcal{E}, \quad (11)$$

where $\mathcal{P}$ denotes the propagation policy space defined in Section 1.3.

The eligibility state associated with revision v is therefore

$$El(v) = f_{\text{elig}}(L(v), Q(v), A(v), I(v), \Pi). \quad (12)$$

Eligibility summarizes the governance status required for operational use. The evaluation itself remains local to the considered revision. Dependency propagation and the computation of effective governance states are introduced separately in the following subsection.

1.5 Propagation Semantics

The governance dimensions introduced in the previous subsections define the local state associated with each ACI revision. Governance evaluation, however, cannot be performed independently for each revision because configuration dependencies may propagate the consequences of a local change throughout the configuration graph. The propagation semantics therefore define how governance states are iteratively updated until a globally consistent evaluation is reached.

Propagation follows the directed relationships of the configuration graph and is controlled by the propagation policies introduced in Section ?? . These policies determine whether a relationship blocks propagation, propagates a warning, carries informational effects only, or does not participate in governance propagation.

The propagation process is defined by a global operator, denoted Prop , which applies the propagation policies over the configuration graph and updates the impact state of affected revisions. Evaluation functions introduced in the previous subsection are subsequently applied locally to derive the resulting governance states.

Formally, let

$$\text{Prop} : G \times \Pi \rightarrow G, \quad (13)$$

where G denotes the governed configuration graph and Π the propagation policy associated with each relationship type.

Starting from an initial governance state, successive applications of Prop generate the sequence

$$G^{(k+1)} = \text{Prop}(G^{(k)}, \Pi), \quad (14)$$

where k denotes the propagation iteration.

Propagation terminates when the governance state becomes stable, that is, when an additional application of the operator no longer modifies any propagated state.

$$\text{Prop}(G^*, \Pi) = G^*. \quad (15)$$

The resulting graph G^* is called the *least fixed point* of the propagation semantics and represents the unique governance state used by the subsequent runtime evaluation.

Because impact propagation is performed before eligibility evaluation, the eligibility of each revision remains a purely local computation. Once the propagated impact state has stabilized, the eligibility state is obtained by applying the evaluation function defined in Section ?? to the final governance descriptor associated with each revision.

The propagation semantics are independent of any implementation strategy. A compliant implementation may therefore employ different execution algorithms, provided that they compute the same least fixed point. The reference implementation adopts a worklist-based evaluation strategy because it avoids re-evaluating unaffected revisions while preserving the normative semantics. The complete worklist algorithm, together with the proofs of monotonicity, termination, convergence, and uniqueness of the least fixed point, is provided in Appendix A.5.

1.5.1 Unified Governance State Diagram

Figure 1 illustrates the interaction between the lifecycle state and the four orthogonal governance state machines.

1.5.2 Transition Properties

The governance state machines satisfy the following properties.

Property G1 (Determinism). For every governance event, each state machine admits exactly one successor state.

Property G2 (Orthogonality). Each governance dimension evolves independently according to its own transition function (see Appendix section ??).

Property G3 (Composability). The global governance state is obtained through the composition of independent semantic dimensions rather than by constructing a single global automaton.

Property G4 (Extensibility). Additional governance dimensions may be introduced without modifying the existing state machines provided that they expose well-defined evaluation functions.

These properties considerably simplify future extensions of ACM while preserving backward compatibility of governance semantics.

The next subsection formalizes how local governance changes propagate through dependency relationships, enabling global reasoning over complete agentic configuration graphs.

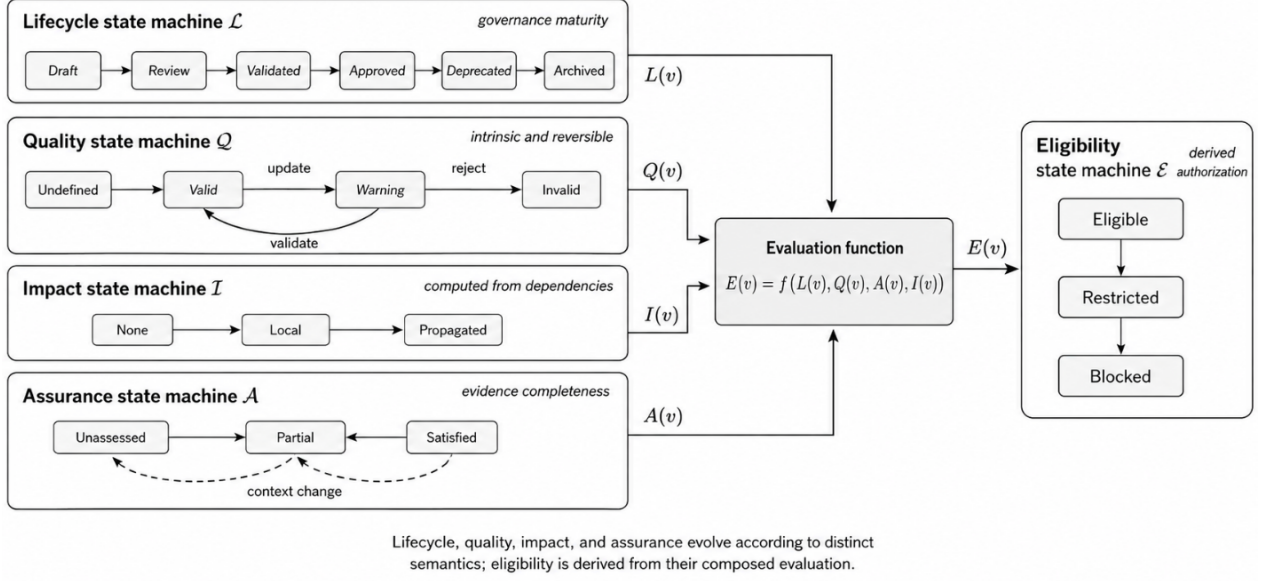


Figure 1: Orthogonal governance state machines composing the ACM governance semantics. Lifecycle, quality, impact, and assurance are evaluated as distinct state dimensions, while execution eligibility is derived through the evaluation function $f(L, Q, A, I)$.

Figure ?? illustrates the propagation of a local configuration change through governance dependencies. The directly modified revision receives the *Local* impact state, while reachable dependent revisions receive the *Propagated* state. Revisions outside the resulting impact closure remain in the *None* state.

1.5.3 Illustrative Example

Assume that a workflow

W

contains an agent

A_1

whose prompt revision is modified. The prompt immediately becomes

Local.

The associated agent, workflow, runtime deployment, and assurance reports become:

Propagated,

because each depends on the modified configuration revision.

Importantly, propagation alone does not imply execution failure.

It merely indicates that governance evaluation should be recomputed before the corresponding artifacts are considered trustworthy.

Figure 3 illustrates the non-destructive propagation of governance state for a single agent. A change to a required prompt revision does not modify the agent revision or its intrinsic lifecycle and quality states. Instead, the propagation process recalculates the agent's impact, assurance, and eligibility states from the updated dependency context.

This fixed-point formulation establishes the formal semantics of governance propagation independently of any particular implementation. Whether ACM is implemented through recursive graph traversal, work-list evaluation, or incremental update algorithms, every compliant implementation is required to compute the same least fixed point, thereby guaranteeing deterministic, reproducible, and implementation-independent governance evaluation.

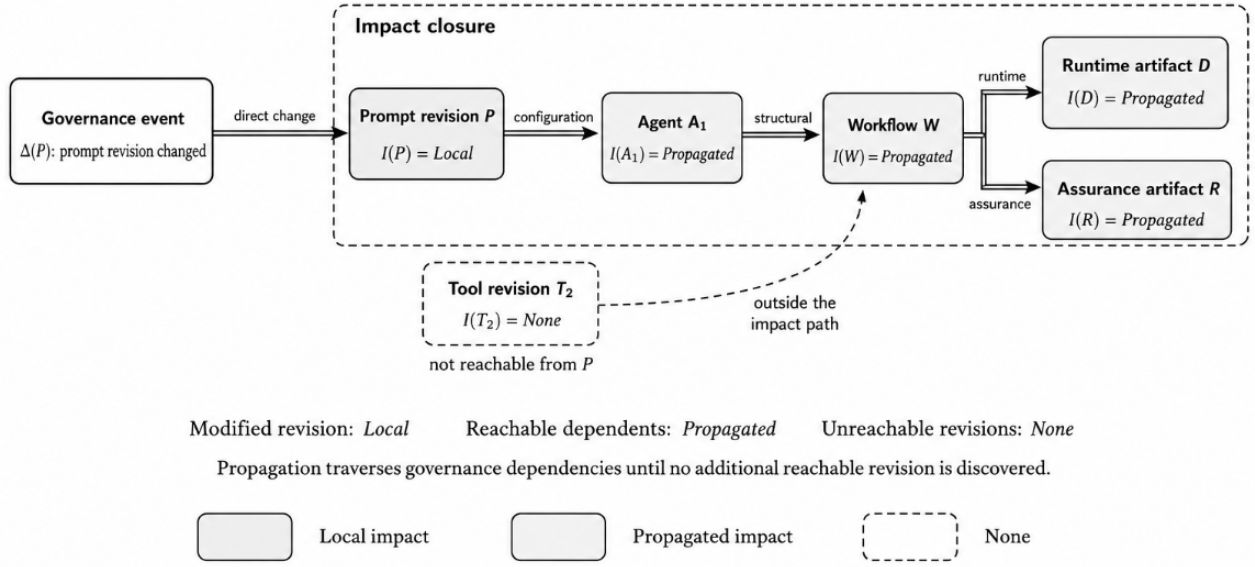


Figure 2: Impact propagation following the modification of prompt revision P . The directly modified revision receives a local impact state, while reachable dependent revisions receive a propagated impact state. Revisions outside the dependency closure remain unaffected.

1.5.4 Illustrative Example

Figure 4 illustrates the propagation of a prompt deprecation through an agentic configuration. The prompt revision becomes **deprecated**; the dependent agent becomes **impacted**; its effective assurance is downgraded to **partial**; and its eligibility changes from **eligible** to **warning**. Importantly, neither the lifecycle nor the declared quality of the agent is modified. This example illustrates the central principle of ACM that governance propagation affects derived governance states rather than intrinsic configuration properties.

1.6 Runtime Semantics

The governance semantics defined so far describe how configuration states evolve within the immutable configuration graph. Runtime execution introduces an additional dimension that cannot be represented solely by static configuration artifacts. Agent executions create transient observations whose purpose is not to modify the governed configuration but to record how a released baseline is instantiated during execution.

Accordingly, ACM distinguishes the *Configuration Graph*, which contains immutable governed revisions, from the *Runtime Graph*, which represents the execution state derived from a specific released baseline. Runtime observations never alter immutable configuration revisions. Instead, they are associated with the corresponding governed artifacts through stable revision identifiers and provenance relationships.

The Runtime Graph is constructed from an ordered sequence of normalized runtime events. Each event captures an observable execution transition together with the identifier of the corresponding governed revision. Because runtime observations are represented independently from framework-specific execution traces, the same governance semantics can be applied to heterogeneous execution frameworks after semantic projection into the ACM reference model.

Rather than storing successive runtime snapshots, ACM defines the runtime state as the result of replaying the normalized event sequence from a released baseline. Starting from the immutable configuration represented by the baseline, runtime events are deterministically applied in chronological order to reconstruct the Runtime Graph corresponding to any observation point.

This separation provides two complementary benefits. First, released configurations remain immutable throughout execution, preserving configuration integrity and provenance. Second, runtime reconstruction becomes independent of the execution framework, provided that equivalent normalized events are available. Consequently, runtime analyses, governance verification, and conformance checking operate on a canonical Runtime Graph rather than on framework-specific execution representations.

The replay semantics are intentionally defined independently of the propagation operator introduced in Section ???. Propagation computes the stabilized governance state associated with immutable configurations,

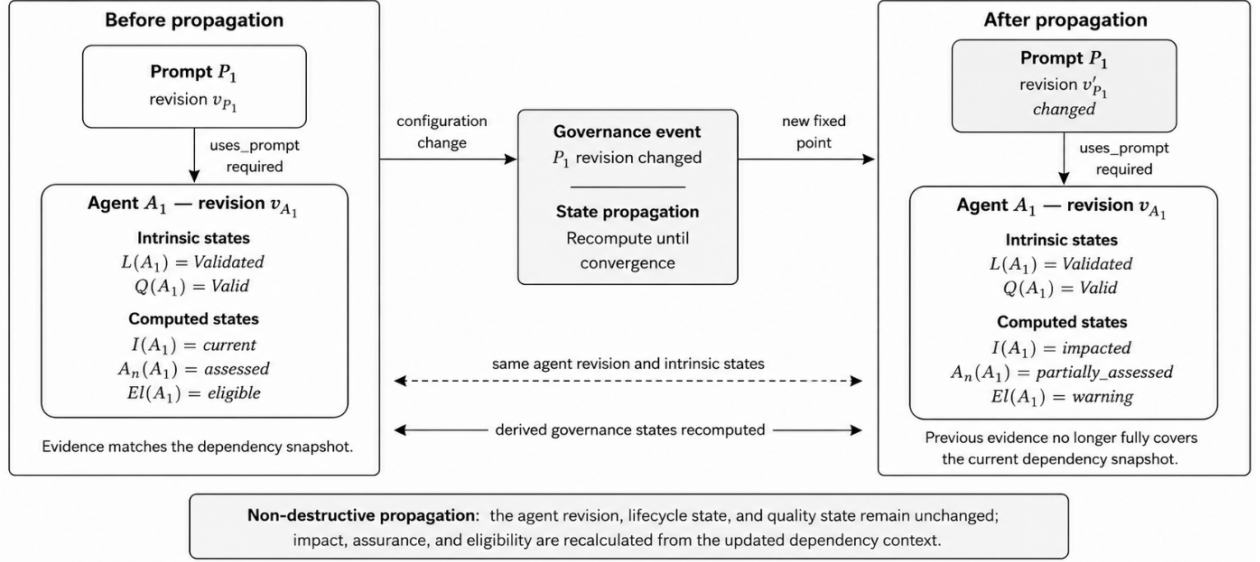


Figure 3: Non-destructive governance state propagation for an agent. A change to a required prompt revision preserves the agent revision and its intrinsic lifecycle and quality states, while the derived impact, assurance, and eligibility states are recomputed from the updated dependency context.

whereas replay reconstructs the execution state generated from a released baseline. These two mechanisms therefore address complementary concerns: governance evaluation determines the valid configuration to be executed, while runtime replay reconstructs how that configuration was actually executed.

The formal definition of normalized runtime events, the replay function, the incremental reconstruction procedure, and the associated correctness properties are presented in Appendix G.

1.7 Formal Properties

The governance semantics defined in the previous subsections satisfy a number of formal properties that ensure the consistency, reproducibility, and predictability of ACM evaluations. These properties follow directly from the state spaces, evaluation functions, propagation semantics, and runtime replay model introduced throughout Section 5.

First, the impact propagation semantics are monotone over the finite impact lattice defined in Appendix A.3. Consequently, iterative propagation converges toward the least fixed point above the initial impact valuation, while the reference worklist algorithm computes the same stabilized result under any fair evaluation strategy. The corresponding proofs are provided in Appendix A.6.

Second, runtime replay is deterministic. Given an immutable released baseline and an identical ordered sequence of normalized runtime events, replay always reconstructs the same Runtime Graph. This property guarantees that runtime observations remain reproducible independently of the underlying execution framework after semantic projection into the ACM reference model. The formal replay model and proof of determinism are presented in Appendix A.7.

Together, these properties establish that governance evaluation and runtime reconstruction are reproducible and implementation-independent. For identical governed configurations and equivalent normalized runtime observations, ACM always produces identical governance states and identical reconstructed Runtime Graphs. This determinism provides the foundation for reproducible governance verification, impact analysis, runtime conformance checking, and assurance evaluation across heterogeneous agentic frameworks.

The complete proofs, together with the complexity analysis and additional formal results, are presented in Appendix A.8.

A Formal Foundations

This appendix provides the mathematical definitions supporting the governance semantics introduced in Section 1. It complements the normative presentation without introducing additional governance concepts.

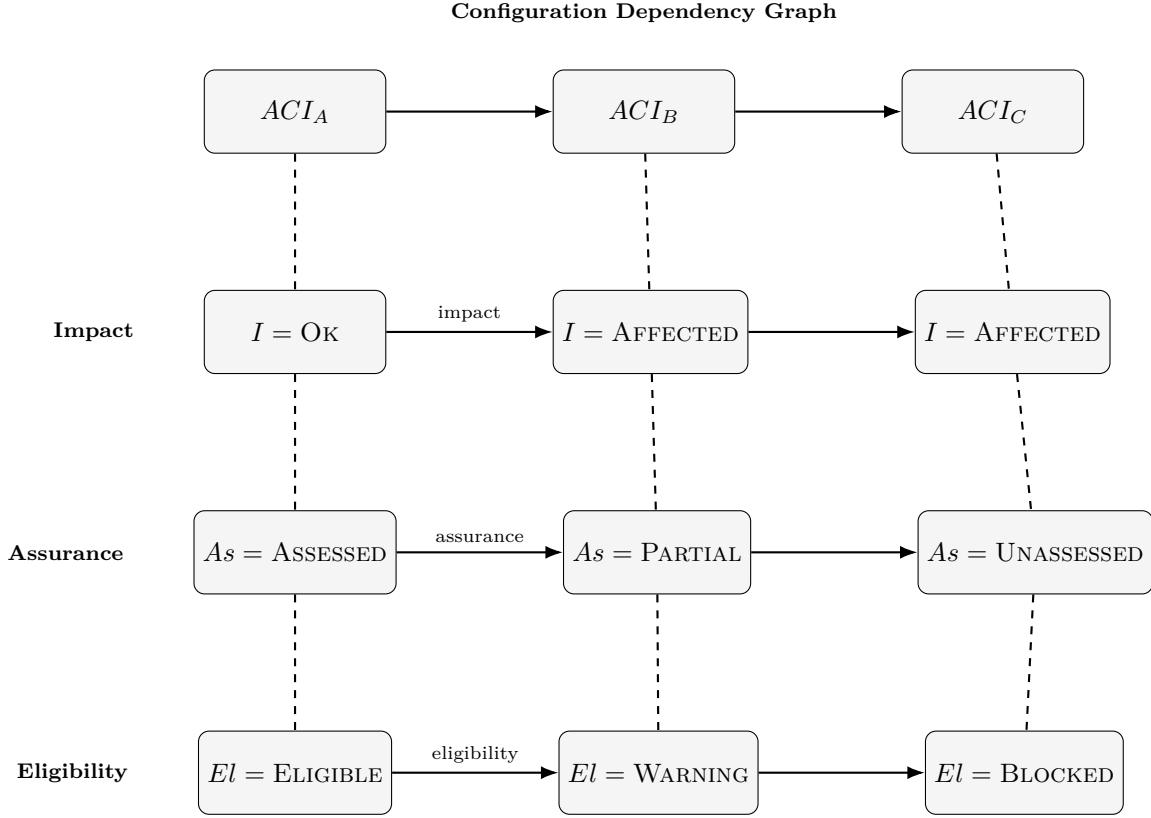


Figure 4: Governance state propagation across the configuration graph. A structural dependency first propagates impact, which subsequently influences assurance evaluation and finally execution eligibility. The three governance dimensions remain conceptually independent while being evaluated in a deterministic order.

A.1 Configuration Graph

The ACM reference model is defined over a finite directed graph

$$G = (V, E), \quad (16)$$

where:

- V denotes the finite set of immutable ACI revisions;
- $E \subseteq V \times V \times T_R$ denotes the finite set of typed governance relationships;
- T_R is the finite set of relationship types defined by the reference metamodel.

The graph topology remains immutable for a given revision set. Configuration evolution is represented by introducing new immutable revisions rather than modifying existing vertices.

A.2 Governance Descriptor

Each revision $v \in V$ is associated with a governance descriptor

$$\Gamma(v) = (L(v), Q(v), A(v), I(v), El(v), Rt(v)), \quad (17)$$

where every component belongs to an independent governance state space.

The governance descriptor represents the complete governance status of an ACI revision at a given observation point. It serves exclusively as a logical aggregation of the different semantic dimensions and introduces no computational semantics.

All semantic computations are defined through dedicated evaluation functions introduced later in this appendix.

A.3 Notation

Throughout the remainder of this appendix, lowercase functions denote state assignments associated with a revision:

$$X : V \rightarrow \mathcal{X}, \quad (18)$$

where $\mathcal{X}$ denotes the corresponding governance state space.

This convention is uniformly applied to lifecycle, quality, assurance, impact, eligibility, and runtime semantics.

For propagation reasoning, an impact valuation is a state assignment

$$\iota : V \rightarrow \mathcal{I}, \quad (19)$$

where $\iota(v)$ denotes the propagated impact state associated with revision v . For a fixed configuration graph, the set of all impact valuations is

$$\mathcal{D}_G = \mathcal{I}^V. \quad (20)$$

The graph structure and the propagation-policy mapping remain fixed during one propagation computation. Only the impact valuation evolves.

B Governance State Spaces

This appendix formally defines the governance state spaces introduced in Section 1.2. Each state space represents an independent semantic dimension.

B.1 Lifecycle

The lifecycle state space is defined as

$$\mathcal{L} = \{\textit{Draft}, \textit{Validated}, \textit{Approved}, \textit{Released}, \textit{Deprecated}, \textit{Archived}\}. \quad (21)$$

The ordering relation is the monotonic lifecycle order introduced in Equation (3).

B.2 Quality

The quality state space is defined as

$$\mathcal{Q} = \{\textit{Undefined}, \textit{Valid}, \textit{Warning}, \textit{Invalid}\}. \quad (22)$$

The quality state reflects the intrinsic validation status of an individual revision independently of its dependencies.

B.3 Assurance

The assurance state space is defined as

$$\mathcal{A} = \{\textit{Unassessed}, \textit{Partial}, \textit{Satisfied}\}. \quad (23)$$

The assurance state represents the availability of governance evidence associated with a revision.

The corresponding ordering relation is

$$\textit{Unassessed} < \textit{Partial} < \textit{Satisfied}. \quad (24)$$

B.4 Impact

The impact state space is defined as

$$\mathcal{I} = \{None, Local, Propagated\}. \quad (25)$$

The impact space is equipped with the total order

$$None \prec_{\mathcal{I}} Local \prec_{\mathcal{I}} Propagated. \quad (26)$$

Its least element is

$$\perp_{\mathcal{I}} = None. \quad (27)$$

The least upper bound of two impact states is defined by

$$i_1 \sqcup_{\mathcal{I}} i_2 = \max_{\prec_{\mathcal{I}}} (i_1, i_2). \quad (28)$$

Consequently, $(\mathcal{I}, \preceq_{\mathcal{I}})$ is a finite lattice.

The impact state identifies whether a revision has been affected by a propagated governance change. —

B.5 Eligibility

The eligibility state space is defined as

$$\mathcal{E} = \{Blocked, Restricted, Eligible\}. \quad (29)$$

The corresponding ordering relation is

$$Blocked < Restricted < Eligible. \quad (30)$$

The eligibility state summarizes whether a revision satisfies the governance conditions required for operational use.

—

B.6 Runtime

The runtime state space is defined as

$$\mathcal{R} = \{Created, Ready, Running, Waiting, Completed, Failed, Cancelled, Terminated\}. \quad (31)$$

The runtime state represents the execution status reconstructed from runtime observations independently of the configuration lifecycle.

Unlike the previous state spaces, runtime states are reconstructed from execution events rather than assigned through governance decisions.

C Propagation Policies

This appendix formally defines the propagation policies introduced in Section 1.3. Propagation policies characterize how governance information traverses configuration relationships independently of the governance state evaluation.

C.1 Policy Space

Let T_R denote the finite set of relationship types defined by the ACM reference model.

Each relationship type is associated with exactly one propagation policy through the mapping

$$\Pi : T_R \rightarrow \mathcal{P}, \quad (32)$$

where the policy space is defined as

$$\mathcal{P} = \{Blocking, Warning, Informational, None\}. \quad (33)$$

The mapping Π is static and depends only on the relationship type. Consequently, identical relationship types always induce identical propagation behavior independently of the connected revisions.

C.2 Policy Precedence

When several propagation paths converge on the same revision, multiple propagation policies may simultaneously apply. Their effective behavior is determined by a total precedence relation over $\mathcal{P}$.

The precedence relation is defined as

$$Blocking > Warning > Informational > None. \quad (34)$$

This ordering reflects increasing governance severity and guarantees deterministic propagation independently of graph traversal order.

C.3 Policy Composition

The effective propagation policy associated with a revision receiving multiple propagated events is obtained by selecting the highest-precedence policy.

The composition operator

$$\otimes : \mathcal{P} \times \mathcal{P} \rightarrow \mathcal{P}, \quad (35)$$

is therefore defined as

$$p_1 \otimes p_2 = \max_{(34)}(p_1, p_2), \quad (36)$$

where the maximum is evaluated according to the precedence relation defined in Equation (34).

By construction, the composition operator satisfies

- commutativity,
- associativity,
- idempotence.

Consequently, policy composition is independent of propagation order, parent ordering, and graph traversal strategy.

These properties ensure that the propagation operator introduced in Appendix E produces deterministic governance results.

Policy precedence determines how concurrent policy contributions are combined. The effect of an individual policy on an impact state is specified separately by the local transfer semantics introduced in Appendix E. Policy composition and impact transfer therefore remain distinct operations: the former selects the effective policy, whereas the latter determines its contribution to impact propagation.

D Formal Evaluation Functions

This appendix formally defines the governance evaluation functions introduced in Section 1.4. These functions assign governance states independently of propagation semantics.

D.1 Quality Evaluation

The quality evaluation function

$$f_{\text{quality}} : V \rightarrow \mathcal{Q}, \quad (37)$$

assigns exactly one quality state to every ACI revision.

Consequently,

$$\forall v \in V, \exists! Q(v) \in \mathcal{Q}. \quad (38)$$

D.2 Assurance Evaluation

Similarly,

$$f_{\text{assurance}} : V \rightarrow \mathcal{A}, \quad (39)$$

assigns a unique assurance state to every revision.

Therefore,

$$\forall v \in V, \exists! A(v) \in \mathcal{A}. \quad (40)$$

D.3 Eligibility Evaluation

Eligibility is evaluated locally from the lifecycle, quality, assurance, and stabilized impact states associated with a revision. Propagation policies do not constitute an argument of the eligibility function; their effects are already reflected in the propagated impact state.

Formally,

$$f_{\text{elig}} : \mathcal{L} \times \mathcal{Q} \times \mathcal{A} \times \mathcal{I} \rightarrow \mathcal{E}. \quad (41)$$

The function is deterministic.

$$\forall (l, q, a, i) \in \mathcal{L} \times \mathcal{Q} \times \mathcal{A} \times \mathcal{I}, \exists! e \in \mathcal{E} \text{ such that } e = f_{\text{elig}}(l, q, a, i). \quad (42)$$

For each revision $v \in V$, eligibility is therefore obtained from the final impact valuation ι^* computed by the propagation semantics:

$$El(v) = f_{\text{elig}}(L(v), Q(v), A(v), \iota^*(v)). \quad (43)$$

Eligibility evaluation is not part of the fixed-point iteration and does not modify the propagated impact valuation.

The evaluation functions are defined as total deterministic mappings over their respective finite domains. They introduce no dependency traversal and do not modify the configuration graph. In particular, eligibility evaluation consumes the stabilized impact valuation produced by propagation but does not participate in the propagation fixed-point computation.

E Propagation Semantics

This appendix formalizes the propagation semantics introduced in Section 1.5. It specifies the propagation operator, the fixed-point semantics, and the reference worklist algorithm. The mathematical properties of the propagation semantics, including monotonicity, termination, convergence, and correctness, are established separately in Appendix H.

E.1 Propagation Operator

Propagation operates over a fixed governed configuration graph

$$G = (V, E),$$

whose structure remains unchanged throughout one propagation computation. Only propagated impact states evolve during the evaluation.

As introduced in Appendix A, an impact valuation is a mapping

$$\iota : V \rightarrow \mathcal{I},$$

where $\iota(v)$ denotes the current propagated impact state associated with revision v .

The propagation process starts from an initial valuation

$$\iota^{(0)} : V \rightarrow \mathcal{I}, \quad (44)$$

which identifies the revisions directly affected by the governance event. Revisions not affected by the initial event are assigned the least impact state

$$\perp_{\mathcal{I}} = \text{None}.$$

The graph structure and the propagation-policy mapping remain fixed throughout the computation. Consequently, propagation modifies only the impact valuation.

Each relationship

$$e = (u, v) \in E$$

is associated with a propagation policy

$$\Pi(e) \in \mathcal{P},$$

defined in Appendix C.

For formal reasoning, each propagation policy induces a local transfer function

$$\tau_p : \mathcal{I} \rightarrow \mathcal{I}, \quad p \in \mathcal{P}, \quad (45)$$

which specifies how the propagated impact state is transmitted across a single relationship governed by policy p .

The precise behavior of each transfer function is determined by the associated propagation policy while preserving the ordering relation defined on $\mathcal{I}$.

For every revision $v \in V$, propagation is defined by the local evaluation function

$$F_v(\iota) = \iota^{(0)}(v) \sqcup_{\mathcal{I}} \bigvee_{(u,v) \in E} \tau_{\Pi(u,v)}(\iota(u)), \quad (46)$$

where $\sqcup_{\mathcal{I}}$ denotes the least upper bound on the impact lattice.

For a revision with no incoming relationship, the empty join is defined as the least impact state:

$$\bigvee_{\emptyset} = \perp_{\mathcal{I}}. \quad (47)$$

The first term preserves the initial impact directly associated with the governance event, whereas the second term aggregates the propagated contributions received from predecessor revisions.

The global propagation operator is therefore defined as

$$\widehat{\text{Prop}}_{G,\Pi} : \mathcal{D}_G \rightarrow \mathcal{D}_G, \quad (48)$$

with

$$\widehat{\text{Prop}}_{G,\Pi}(\iota)(v) = F_v(\iota), \quad \forall v \in V. \quad (49)$$

The operator acts exclusively on impact valuations. Lifecycle, quality, assurance, and eligibility states are not modified during propagation. Once the impact valuation has stabilized, eligibility is evaluated locally using the function f_{elig} defined in Appendix D.

For readability, the remainder of this appendix denotes $\widehat{\text{Prop}}_{G,\Pi}$ simply by $\widehat{\text{Prop}}$ whenever the underlying graph and propagation-policy mapping are fixed.

E.2 Least Fixed-Point Semantics

Starting from the initial impact valuation $\iota^{(0)}$, successive applications of the propagation operator generate the sequence

$$\iota^{(k+1)} = \widehat{\text{Prop}}(\iota^{(k)}), \quad (50)$$

where k denotes the propagation iteration.

Propagation continues until the impact valuation becomes stable, that is, until an additional application of the propagation operator produces no further modification.

The stabilized valuation therefore satisfies

$$\iota^* = \widehat{\text{Prop}}(\iota^*). \quad (51)$$

The valuation ι^* represents the final propagated impact state associated with every revision of the configuration graph.

Once the impact valuation has stabilized, eligibility is evaluated independently for every revision using the local evaluation function defined in Appendix D. Formally,

$$El(v) = f_{\text{elig}}(L(v), Q(v), A(v), \iota^*(v)), \quad \forall v \in V. \quad (52)$$

Consequently, the propagation phase computes only the propagated impact valuation, whereas eligibility evaluation is performed afterwards as an independent local computation. This separation eliminates any circular dependency between propagation and eligibility evaluation while preserving the normative semantics introduced in Section 1.5.

The existence, uniqueness, and convergence properties of the stabilized impact valuation, together with the mathematical justification of the fixed-point construction, are established in Appendix H.

E.3 Reference Worklist Algorithm

The propagation semantics specify the stable impact valuation independently of the evaluation strategy used to compute it. Any implementation producing the fixed point defined by Equation (51) is therefore compliant with the reference semantics.

The reference implementation adopts a worklist-based evaluation strategy that incrementally updates only the revisions whose propagated impact state may change.

Initially, the current valuation is set to the initial impact valuation

$$\iota = \iota^{(0)},$$

and the worklist

$$W \subseteq V$$

contains every revision directly affected by the triggering governance event, together with their immediate successors.

At each iteration, one revision

$$v \in W$$

is removed from the worklist and its local propagation equation (Equation (46)) is evaluated using the current impact valuation.

If the evaluation produces a strictly greater impact state,

$$F_v(\iota) \succ_{\mathcal{I}} \iota(v), \quad (53)$$

the valuation is updated according to

$$\iota(v) := F_v(\iota), \quad (54)$$

and every successor

$$w \text{ such that } (v, w) \in E$$

is inserted into the worklist if not already present.

Otherwise, no successor is scheduled and propagation continues with the next revision contained in the worklist.

The evaluation terminates when the worklist becomes empty,

$$W = \emptyset. \quad (55)$$

The algorithm terminates operationally when the worklist becomes empty:

$$W = \emptyset. \quad (56)$$

Under the worklist initialization, successor rescheduling, bottom-preserving transfer semantics, and fairness conditions defined above, an empty worklist corresponds to a valuation satisfying every local propagation equation. The formal implication

$$W = \emptyset \implies \forall v \in V, F_v(\iota) = \iota(v) \quad (57)$$

and the equality of the resulting valuation with ι^* are established in Appendix A.6.

The reference worklist algorithm assumes a fair evaluation strategy, meaning that every revision inserted into the worklist is eventually processed. Different scheduling strategies, including FIFO, LIFO, or priority-based queues, remain compliant provided that they satisfy this fairness condition and compute the same stabilized valuation.

The correctness, termination, convergence, and complexity of the reference worklist algorithm are established in Appendix H.

F Formal Properties of the Propagation Semantics

This appendix establishes the mathematical properties of the propagation semantics introduced in Appendix E. It proves the correctness of the propagation operator, the existence and uniqueness of the least fixed point, the convergence of the iterative evaluation, and the correctness of the reference worklist algorithm. Throughout this appendix, the configuration graph and the propagation-policy mapping are assumed to be fixed.

F.1 Formal Properties of the Propagation Domain

The propagation operator acts on the impact valuation domain

$$\mathcal{D}_G = \mathcal{I}^V,$$

introduced in Appendix A, where each valuation

$$\iota : V \rightarrow \mathcal{I}$$

associates one propagated impact state with every revision of the fixed configuration graph.

The ordering relation on $\mathcal{D}_G$ is defined componentwise.

$$\iota_1 \sqsubseteq \iota_2 \iff \forall v \in V, \iota_1(v) \preceq_{\mathcal{I}} \iota_2(v). \quad (58)$$

Since both the configuration graph and the impact state space are finite (Appendices A and B), the valuation domain $\mathcal{D}_G$ is finite.

Moreover, because $(\mathcal{I}, \preceq_{\mathcal{I}})$ is a finite lattice, the product domain $(\mathcal{D}_G, \sqsubseteq)$ is itself a finite lattice under the componentwise ordering, whose least element is the valuation

$$\perp_{\mathcal{D}_G}(v) = \perp_{\mathcal{I}}, \quad \forall v \in V. \quad (59)$$

The least upper bound of two valuations is defined componentwise by

$$(\iota_1 \sqcup \iota_2)(v) = \iota_1(v) \sqcup_{\mathcal{I}} \iota_2(v), \quad \forall v \in V. \quad (60)$$

The finite lattice $(\mathcal{D}_G, \sqsubseteq)$ constitutes the mathematical domain on which the propagation operator $\widehat{\text{Prop}}$ is defined.

F.2 Monotonicity of Local Transfer Functions

The propagation operator introduced in Appendix E is constructed from the local transfer functions

$$\tau_p : \mathcal{I} \rightarrow \mathcal{I}, \quad p \in \mathcal{P},$$

associated with the propagation policies.

By construction, each transfer function preserves the ordering relation defined on the impact lattice.

Formally,

$$\forall x, y \in \mathcal{I}, \quad x \preceq_{\mathcal{I}} y \implies \tau_p(x) \preceq_{\mathcal{I}} \tau_p(y). \quad (61)$$

In particular,

$$\tau_p(\perp_{\mathcal{I}}) = \perp_{\mathcal{I}}, \quad \forall p \in \mathcal{P}. \quad (62)$$

Consequently, a relationship cannot generate a propagated impact in the absence of an impacted predecessor. The transfer functions may transmit or suppress an existing impact according to the associated propagation policy, but they do not create impact independently of the initial valuation. The local propagation equation associated with a revision

$$v \in V$$

is

$$F_v(\iota) = \iota^{(0)}(v) \sqcup_{\mathcal{I}} \bigvee_{(u,v) \in E} \tau_{\Pi(u,v)}(\iota(u)).$$

Monotonicity of the local propagation equation. Let

$$\iota_1 \sqsubseteq \iota_2.$$

By definition of the product ordering,

$$\iota_1(u) \preceq_{\mathcal{I}} \iota_2(u) \quad \forall u \in V.$$

Applying Equation (61) to every incoming relationship yields

$$\tau_{\Pi(u,v)}(\iota_1(u)) \preceq_{\mathcal{I}} \tau_{\Pi(u,v)}(\iota_2(u)).$$

Since the least upper bound operator $\sqcup_{\mathcal{I}}$ is monotone on the finite lattice $\mathcal{I}$, the aggregation of propagated contributions also preserves the ordering.

Therefore,

$$F_v(\iota_1) \preceq_{\mathcal{I}} F_v(\iota_2), \quad \forall v \in V. \quad (63)$$

Consequently, every local propagation equation is monotone with respect to the componentwise ordering defined on the impact valuation domain.

F.3 Global Monotonicity of the Propagation Operator

The global propagation operator is obtained by assembling the local propagation equations associated with each revision of the fixed configuration graph.

Formally,

$$\widehat{\text{Prop}}(\iota) = (F_v(\iota))_{v \in V}, \quad (64)$$

where each local equation F_v is defined as in Appendix F.2.

Proposition. The global propagation operator

$$\widehat{\text{Prop}} : \mathcal{D}_G \rightarrow \mathcal{D}_G$$

is monotone with respect to the product ordering $\sqsubseteq$.

Proof. Let

$$\iota_1 \sqsubseteq \iota_2.$$

By Appendix F.2, every local propagation equation is monotone, that is,

$$F_v(\iota_1) \preceq_{\mathcal{I}} F_v(\iota_2), \quad \forall v \in V.$$

Since the product ordering on

$$\mathcal{D}_G = \mathcal{I}^V$$

is defined componentwise, the monotonicity of every component immediately implies

$$\widehat{\text{Prop}}(\iota_1) \sqsubseteq \widehat{\text{Prop}}(\iota_2).$$

Therefore,

$$\iota_1 \sqsubseteq \iota_2 \implies \widehat{\text{Prop}}(\iota_1) \sqsubseteq \widehat{\text{Prop}}(\iota_2). \quad (65)$$

Hence, the propagation operator is monotone over the finite lattice $\mathcal{D}_G$.

F.4 Existence of the Least Fixed Point

The previous results establish that the propagation domain

$$\mathcal{D}_G = \mathcal{I}^V$$

is a finite lattice (Appendix F.1) and that the global propagation operator

$$\widehat{\text{Prop}} : \mathcal{D}_G \rightarrow \mathcal{D}_G$$

is monotone (Appendix F.3).

The classical Knaster–Tarski Fixed Point Theorem therefore applies directly.

Theorem (Knaster–Tarski). For the propagation event represented by $\iota^{(0)}$, consider the principal upper set

$$\mathcal{D}_G^{\iota^{(0)}} = \left\{ \iota \in \mathcal{D}_G \mid \iota^{(0)} \sqsubseteq \iota \right\}. \quad (66)$$

Because $\mathcal{D}_G$ is a finite lattice, $\mathcal{D}_G^{\iota^{(0)}}$ is itself a finite complete lattice whose least element is $\iota^{(0)}$. Moreover, the preservation of the initial valuation in every local propagation equation implies

$$\iota^{(0)} \sqsubseteq \widehat{\text{Prop}}(\iota), \quad \forall \iota \in \mathcal{D}_G^{\iota^{(0)}}. \quad (67)$$

Hence, $\widehat{\text{Prop}}$ maps $\mathcal{D}_G^{\iota^{(0)}}$ into itself. Since the operator is monotone, the Knaster–Tarski theorem guarantees the existence of a least fixed point in this restricted domain.

This fixed point is denoted by ι^* and satisfies

$$\widehat{\text{Prop}}(\iota^*) = \iota^*. \quad (68)$$

It is characterized by

$$\iota^* = \bigwedge \left\{ \iota \in \mathcal{D}_G^{\iota^{(0)}} \mid \widehat{\text{Prop}}(\iota) = \iota \right\}. \quad (69)$$

Therefore, ι^* is the unique least fixed point above $\iota^{(0)}$, rather than necessarily the least fixed point of the operator over the entire valuation domain.

F.5 Convergence by Increasing Iteration

The existence of a least fixed point does not by itself establish that the iterative evaluation defined in Appendix E reaches that fixed point. This subsection establishes the increasing character, finite stabilization, and minimality of the propagation sequence.

Starting from the initial impact valuation $\iota^{(0)}$, the sequence is defined recursively by

$$\iota^{(k+1)} = \widehat{\text{Prop}}(\iota^{(k)}), \quad k \geq 0. \quad (70)$$

Increasing character of the propagation sequence. For every revision $v \in V$, the local propagation equation gives

$$F_v(\iota^{(0)}) = \iota^{(0)}(v) \sqcup_{\mathcal{I}} \bigvee_{(u,v) \in E} \tau_{\Pi(u,v)}(\iota^{(0)}(u)). \quad (71)$$

Since each operand of a join is lower than or equal to the resulting least upper bound,

$$\iota^{(0)}(v) \preceq_{\mathcal{I}} F_v(\iota^{(0)}), \quad \forall v \in V. \quad (72)$$

By the componentwise ordering on $\mathcal{D}_G$, it follows that

$$\iota^{(0)} \sqsubseteq \widehat{\text{Prop}}(\iota^{(0)}) = \iota^{(1)}. \quad (73)$$

Assume now that, for some $k \geq 0$,

$$\iota^{(k)} \sqsubseteq \iota^{(k+1)}. \quad (74)$$

By the global monotonicity of $\widehat{\text{Prop}}$ established in the previous subsection,

$$\widehat{\text{Prop}}(\iota^{(k)}) \sqsubseteq \widehat{\text{Prop}}(\iota^{(k+1)}). \quad (75)$$

Using Equation (70), this is equivalent to

$$\iota^{(k+1)} \sqsubseteq \iota^{(k+2)}. \quad (76)$$

Therefore, by induction, the propagation sequence forms an increasing chain:

$$\iota^{(0)} \sqsubseteq \iota^{(1)} \sqsubseteq \dots \sqsubseteq \iota^{(k)} \sqsubseteq \iota^{(k+1)} \sqsubseteq \dots. \quad (77)$$

Finite stabilization. The valuation domain $\mathcal{D}_G$ is finite. Every increasing chain in $\mathcal{D}_G$ must therefore become stationary after a finite number of iterations. Hence, there exists an integer $N \geq 0$ such that

$$\iota^{(N)} = \iota^{(N+1)}. \quad (78)$$

By Equation (70),

$$\widehat{\text{Prop}}(\iota^{(N)}) = \iota^{(N)}. \quad (79)$$

Thus, the stabilized valuation is a fixed point of the propagation operator.

Minimality of the stabilized valuation. Let $\mu \in \mathcal{D}_G$ be any fixed point of $\widehat{\text{Prop}}$ satisfying

$$\iota^{(0)} \sqsubseteq \mu. \quad (80)$$

It follows by induction that

$$\iota^{(k)} \sqsubseteq \mu, \quad \forall k \geq 0. \quad (81)$$

The base case follows from Equation (80). For the induction step, assume $\iota^{(k)} \sqsubseteq \mu$. Global monotonicity and the fixed-point property of μ yield

$$\iota^{(k+1)} = \widehat{\text{Prop}}(\iota^{(k)}) \sqsubseteq \widehat{\text{Prop}}(\mu) = \mu. \quad (82)$$

In particular,

$$\iota^{(N)} \sqsubseteq \mu. \quad (83)$$

Consequently, $\iota^{(N)}$ is the least fixed point of the propagation operator above the initial valuation. Therefore,

$$\iota^{(N)} = \iota^*. \quad (84)$$

The propagation iteration thus converges after finitely many steps to the unique least stable impact valuation consistent with the initial impact assignment and the propagation policies.

F.6 Correctness of the Reference Worklist Algorithm

The previous subsections establish that the propagation operator admits a unique least fixed point above the initial impact valuation and that the iterative propagation semantics converge toward this valuation. This subsection shows that the reference worklist algorithm computes exactly the same result.

The worklist algorithm performs asynchronous evaluations of the local propagation equations defined in Equation (46). Only revisions whose propagated impact state may change are scheduled for re-evaluation.

Soundness. Assume that the worklist algorithm terminates with the valuation

$$\iota.$$

By definition of the algorithm, the worklist is empty,

$$W = \emptyset.$$

Furthermore, every revision whose local propagation equation changes is immediately reinserted into the worklist. Consequently, an empty worklist implies that no local propagation equation can further increase the current valuation.

Therefore,

$$F_v(\iota) = \iota(v), \quad \forall v \in V.$$

Using the definition of the global propagation operator,

$$\widehat{\text{Prop}}(\iota) = (F_v(\iota))_{v \in V},$$

it follows immediately that

$$\widehat{\text{Prop}}(\iota) = \iota. \tag{85}$$

Hence, every valuation returned by the worklist algorithm is a fixed point of the propagation operator.

Completeness. The worklist algorithm starts from the initial impact valuation $\iota^{(0)}$ and applies exactly the same local propagation equations as the global iterative evaluation.

Each update replaces the current valuation of one revision by the value prescribed by its local propagation equation while leaving every other component unchanged.

Since every scheduled update is monotone (Appendix F.2), the sequence of valuations generated by the worklist algorithm remains increasing.

Moreover, the fairness assumption introduced in Appendix E guarantees that every revision whose propagated impact state may still change is eventually processed.

Consequently, no local propagation equation capable of modifying the valuation can remain permanently unapplied.

The stabilized valuation computed by the worklist algorithm therefore satisfies all local propagation equations and is reachable from the initial valuation through a finite sequence of monotone updates.

Since Appendix ?? establishes the uniqueness of the least fixed point above $\iota^{(0)}$, the valuation produced by the worklist algorithm must coincide with this least fixed point.

Therefore,

$$\iota = \iota^*. \tag{86}$$

The reference worklist algorithm is therefore correct: it computes exactly the least fixed point defined by the propagation semantics while avoiding the unnecessary global re-evaluation of unaffected revisions.

F.7 Termination and Complexity

The previous subsections establish that the propagation sequence forms an increasing chain in the finite lattice $\mathcal{D}_G$ and that the reference worklist algorithm computes the same least fixed point as the global propagation operator.

Termination. Each update performed by the worklist algorithm replaces the current impact state of a revision only when the corresponding local propagation equation produces a strictly greater value with respect to the ordering $\preceq_{\mathcal{I}}$.

Consequently, every successful update strictly increases one component of the current impact valuation while leaving all other components unchanged.

Since the impact lattice $\mathcal{I}$ is finite, every revision can undergo only a finite number of strictly increasing updates.

Furthermore, the configuration graph contains a finite number of revisions.

Therefore, only finitely many successful updates can occur during one propagation computation.

Because every unsuccessful evaluation removes one revision from the worklist without modifying the current valuation, the worklist eventually becomes empty.

Hence, the reference worklist algorithm always terminates after a finite number of iterations.

Complexity. Let

$$n = |V|, \quad m = |E|,$$

and let

$$h_{\mathcal{I}}$$

denote the height of the impact lattice.

Since each revision can experience at most $h_{\mathcal{I}} - 1$ strictly increasing updates, each outgoing relationship is evaluated at most $h_{\mathcal{I}}$ times.

The overall propagation complexity is therefore bounded by

$$O(h_{\mathcal{I}} m). \quad (87)$$

For the three-level impact lattice introduced in Appendix B,

$$\mathcal{I} = \{None, Local, Propagated\},$$

the lattice height is

$$h_{\mathcal{I}} = 3.$$

Consequently, each revision can undergo at most two strictly increasing impact updates, and the propagation complexity becomes

$$O(3m) = O(m). \quad (88)$$

Thus, propagation is linear in the number of configuration relationships for the reference ACM impact model.

G Runtime Semantics

This appendix formalizes the runtime semantics introduced in Section ?? . Runtime semantics are intentionally defined independently from the propagation operator. Whereas propagation computes stabilized governance states over immutable configuration revisions, runtime semantics reconstruct execution states from normalized runtime events. Throughout this appendix, the released configuration baseline remains fixed and immutable.

G.1 Runtime Event Model

Let

$$B = (V_B, E_B)$$

denote a released configuration baseline.

A runtime execution is represented by a finite ordered sequence of normalized runtime events

$$\Sigma = \langle e_1, e_2, \dots, e_n \rangle. \quad (89)$$

Each event is represented by the tuple

$$e = (id, t, r, \alpha, m), \quad (90)$$

where

- id uniquely identifies the event,
- t is its logical timestamp,
- r identifies the immutable ACI revision concerned by the event,
- α denotes the event type,
- m contains optional execution metadata.

The event sequence is assumed to be totally ordered by timestamp.

Each event references exactly one immutable revision already present in the released baseline. Runtime events therefore introduce execution observations without modifying the governed configuration.

G.2 Replay Function

Let

$$R_0 = B$$

denote the initial Runtime Graph obtained directly from the released baseline.

A replay step is defined by

$$Replay : \mathcal{G} \times \mathcal{E}_r \rightarrow \mathcal{G}, \quad (91)$$

where $\mathcal{G}$ denotes the set of runtime graphs and $\mathcal{E}_r$ the set of normalized runtime events.

Applying an event updates only the runtime representation associated with the referenced immutable revision.

The complete runtime reconstruction is therefore defined recursively as

$$R_i = Replay(R_{i-1}, e_i), \quad 1 \leq i \leq n. \quad (92)$$

The reconstructed Runtime Graph after processing the complete execution trace is

$$R_n.$$

The replay function never modifies immutable configuration revisions and never creates governance information absent from the released baseline.

G.3 Incremental Runtime Reconstruction

Because replay is defined event by event, runtime reconstruction is naturally incremental.

Let

$$\Sigma_k = \langle e_1, \dots, e_k \rangle \quad (93)$$

denote the prefix of length k .

The Runtime Graph reconstructed after observing this prefix is

$$R_k = Replay^*(B, \Sigma_k), \quad (94)$$

where $Replay^*$ denotes repeated application of the replay function.

Consequently, extending an execution trace requires replaying only newly observed events,

$$R_{k+1} = Replay(R_k, e_{k+1}), \quad (95)$$

without rebuilding the Runtime Graph from scratch.

This property enables efficient online reconstruction while preserving exactly the same semantics as complete replay.

G.4 Provenance Preservation

Every runtime observation remains associated with the immutable revision from which it originates.

Formally, let

$$prov : V_R \rightarrow V_B \quad (96)$$

associate each runtime node with its originating configuration revision.

For every runtime node

$$x \in V_R,$$

the provenance mapping satisfies

$$prov(x) \in V_B. \quad (97)$$

Consequently,

- immutable revision identities remain unchanged;

- runtime observations never duplicate governed revisions;
- every runtime state can be traced back to a unique immutable configuration revision.

Runtime replay therefore preserves the complete provenance chain established by the released baseline.

G.5 Determinism of Runtime Replay

Runtime replay is deterministic under the following assumptions:

1. the released baseline is fixed;
2. the normalized runtime event sequence is identical;
3. events are replayed in the same total order.

Under these assumptions,

$$\text{Replay}^*(B, \Sigma)$$

always produces the same Runtime Graph.

Proof. The proof proceeds by induction on the length of the event sequence.

The base case is immediate since replay over the empty sequence returns the released baseline.

Assume the property holds for the first k events.

Since the replay function is deterministic and event ordering is fixed, processing event e_{k+1} produces a uniquely determined successor Runtime Graph.

Therefore the reconstructed Runtime Graph is uniquely determined after every replay step.

By induction, replay deterministically reconstructs the Runtime Graph for the complete event sequence.

G.6 Reconstruction Properties

The runtime reconstruction defined above satisfies the following properties.

Baseline preservation Replay never modifies immutable configuration revisions.

Provenance preservation Every runtime observation remains associated with exactly one immutable configuration revision.

Incrementality Replaying an additional event requires updating only the previously reconstructed Runtime Graph.

Determinism Identical released baselines and identical normalized event sequences always produce identical Runtime Graphs.

Framework independence Because replay operates exclusively on normalized runtime events, the reconstruction semantics are independent of the native execution framework once semantic projection into the ACM reference model has been completed.

H Formal Properties

H.1 Reproducibility of Governance Evaluation

The governance semantics defined throughout this paper are fully deterministic once the governed configuration, propagation policies, and runtime observations are fixed. Consequently, governance evaluation is reproducible across repeated executions.

More precisely, let

$$B$$

be a released baseline,

Π

the propagation policy assignment,
and

Σ

a normalized runtime event sequence.

The governance evaluation associated with these inputs is uniquely determined by

$$\mathcal{G}(B, \Pi, \Sigma),$$

which denotes the complete governance state obtained after impact propagation, local evaluation, and runtime replay.

Proposition. For identical inputs

$$(B, \Pi, \Sigma),$$

ACM always produces the same governance evaluation.

Proof. The result follows directly from the previous appendices.

Impact propagation converges toward a unique least fixed point (Appendix ??).

The evaluation functions f_{quality} , $f_{\text{assurance}}$, and f_{elig} are deterministic local functions.

Runtime replay is deterministic for a fixed released baseline and an identical ordered event sequence (Appendix G.5).

Since each stage of the evaluation pipeline is deterministic, their composition is also deterministic.

Therefore identical inputs always produce identical governance states. $\square$

H.2 Framework-Independent Consistency

The ACM governance semantics are intentionally defined over the normalized reference model rather than over framework-specific execution representations.

Assume two heterogeneous execution frameworks

$$F_1 \quad \text{and} \quad F_2,$$

whose native configurations are projected onto the ACM reference model through their respective semantic adapters.

Let

$$Proj(F_1) = Proj(F_2) = B,$$

meaning that both native configurations produce the same normalized released baseline.

Similarly, let

$$Norm(\Sigma_1) = Norm(\Sigma_2),$$

where $Norm$ denotes the runtime normalization process.

Proposition. Equivalent normalized configurations and equivalent normalized runtime event sequences always produce identical governance evaluations, independently of the original execution framework.

Proof. After semantic projection, governance evaluation depends exclusively on

$$(B, \Pi, \Sigma),$$

which belong entirely to the ACM reference model.

Framework-specific implementation details are eliminated during semantic projection and therefore do not participate in the governance semantics.

Since propagation, evaluation, and runtime replay depend only on the normalized representation, two equivalent normalized inputs necessarily produce identical governance states.

Consequently, governance evaluation is independent of the original execution framework, provided that semantic projection preserves the governance-relevant information represented by the ACM reference model. $\square$

H.3 Determinism of the Complete Governance Semantics

The previous appendices establish the determinism of each individual stage of the ACM governance semantics.

Impact propagation converges toward a unique stabilized impact valuation (Appendix ??).

The governance evaluation functions are deterministic by construction, since they operate exclusively on local governance states.

Runtime replay deterministically reconstructs the Runtime Graph from an immutable released baseline and an ordered sequence of normalized runtime events (Appendix G.5).

These results immediately imply the following property.

Theorem. Let

$$(B, \Pi, \Sigma)$$

be a released baseline, a propagation policy assignment, and a normalized runtime event sequence.

The complete ACM governance semantics produces a unique governance evaluation.

Proof. The complete governance semantics is obtained by composing deterministic functions:

$$\textit{Propagation} \circ \textit{Evaluation} \circ \textit{Replay}.$$

Since each component is deterministic under the hypotheses established in the previous appendices, their composition is deterministic.

Therefore the complete governance evaluation is uniquely determined. $\square$

H.4 Complexity Analysis

Let

$$|V|$$

denote the number of immutable revisions,

$$|E|$$

the number of dependency relationships,
and

$$h_{\mathcal{I}}$$

the height of the impact lattice.

As established in Appendix ??, each revision may undergo at most

$$h_{\mathcal{I}} - 1$$

strict impact increases.

Consequently, the reference worklist algorithm performs at most

$$O(h_{\mathcal{I}}|E|)$$

successful propagation updates.

Since the impact lattice adopted by ACM is finite and consists of the ordered states

$$None < Local < Propagated,$$

its height is

$$h_{\mathcal{I}} = 3.$$

Therefore the propagation complexity becomes

$$O(|E|).$$

Runtime replay processes each normalized event exactly once.
For an execution trace containing

$$|\Sigma|$$

events, replay therefore requires

$$O(|\Sigma|)$$

time.

The overall governance evaluation consequently remains linear with respect to the size of the dependency graph and the observed runtime trace.

H.5 Height of the Product Lattice

The convergence proof presented in Appendix ?? relies on the finiteness of the impact valuation domain.

Let

$$\mathcal{D}_G = \mathcal{I}^V$$

denote the product lattice associated with the configuration graph.
Because the impact lattice has height

$$h_{\mathcal{I}},$$

the height of the product lattice satisfies

$$H_G = 1 + |V| (h_{\mathcal{I}} - 1). \quad (98)$$

For the impact lattice adopted by ACM,

$$h_{\mathcal{I}} = 3,$$

yielding

$$H_G = 1 + 2|V|.$$

This finite upper bound guarantees that every strictly increasing propagation sequence eventually stabilizes, providing the theoretical foundation for the termination of both the increasing iteration and the reference worklist algorithm established in Appendix A.6.

I Operationalization: Projection Formalism

I.1 Native Structure Extraction

Let

$$F$$

denote an agentic framework and let

$$M_F$$

denote a native framework model extracted from a particular agentic system.

The extracted model is represented as

$$M_F = (N_F, R_F, P_F), \quad (99)$$

where

- N_F is the set of native framework entities;
- R_F is the set of native relationships between these entities;
- P_F denotes their framework-specific properties and metadata.

Depending on the framework, native entities may include agents, workflow nodes, tasks, tools, prompts, language-model configurations, routing conditions, memory components, or execution policies.

Extraction is performed by a framework-specific projection adapter. The adapter does not assign governance meaning directly. Its responsibility is limited to identifying native objects and recovering the structural and configuration information exposed by the framework.

This distinction prevents framework-specific implementation concepts from entering the ACM governance kernel.

I.2 Semantic Classification

The extracted native entities are subsequently classified according to the ACI taxonomy defined in Section ??.

Let

$$\tau_F : N_F \rightarrow T_{\text{ACI}} \cup \{\perp\} \quad (100)$$

be the semantic classification function, where

$$T_{\text{ACI}} \quad (101)$$

is the set of ACM configuration-item types and

$$\tau_F(n) = \perp \quad (102)$$

indicates that the native entity n has no direct canonical representation in the current ACM model.

A native entity is classified as an ACI only when it satisfies at least one of the following conditions:

- it contributes to the declared structure of the agentic system;
- it affects execution behaviour;
- it carries policy, permission, assurance, or deployment information;
- it must be versioned to support reproducibility or auditability.

Native implementation artifacts that do not carry configuration significance may be deliberately excluded. Such exclusions must nevertheless remain observable through the coverage analysis introduced below.

Native relationships are classified independently through

$$\rho_F : R_F \rightarrow T_R \cup \{\perp\}, \quad (103)$$

where T_R is the set of canonical ACM relationship types.

This independent treatment is necessary because successful projection of two entities does not imply that the semantic relationship connecting them has also been preserved.

I.3 Canonical Normalization

Once classified, native entities and relationships are normalized into the ACM canonical representation.

The entity projection function is defined as

$$\phi_F : N_F \rightarrow V_c, \quad (104)$$

where

$$V_c$$

is the vertex set of the resulting Configuration Graph

$$G_c = (V_c, E_c).$$

The partial nature of ϕ_F reflects that not every native entity is necessarily representable by the current ACM reference model.

For every entity $n \in N_F$ such that

$$\tau_F(n) \neq \perp,$$

the projection produces an immutable ACI revision containing:

- a canonical ACI type;
- a stable logical identifier;
- a revision identifier;
- a content digest;
- normalized configuration attributes;
- provenance linking the ACI revision to the native object.

Similarly, projected relationships are obtained through

$$\psi_F : R_F \rightarrow E_c. \quad (105)$$

Each canonical relationship records its type, source and target ACIs, its cardinality constraints, and the governance metadata required by the propagation semantics.

Normalization therefore produces a framework-independent graph over which the governance kernel can apply lifecycle validation, impact propagation, assurance evaluation, and eligibility computation.

I.4 Projection Traceability

Every projected ACI retains explicit provenance to its source framework object.

Let

$$src : V_c \rightarrow N_F \quad (106)$$

associate each canonical ACI with the native entity from which it was derived.

For every successfully projected entity,

$$src(\phi_F(n)) = n. \quad (107)$$

A corresponding mapping is maintained for relationships.

This bidirectional traceability serves three purposes. First, it allows the projection result to be audited against the original framework definition. Second, it identifies unsupported or partially represented constructs. Third, it allows a governance finding produced by ACM to be related back to the native framework object affected by that finding.

I.4.1 Projection Coverage

Projection expressiveness cannot be assessed solely by demonstrating that a framework can be translated into ACM. It is also necessary to quantify how much of the source framework model is represented by the projection.

Let

$$N_F^{\text{rel}} \subseteq N_F \quad (108)$$

denote the set of governance-relevant native entities considered within the scope of the adapter evaluation. Entity Projection Coverage is defined as

$$C_N(F) = \frac{|\{n \in N_F^{\text{rel}} \mid \phi_F(n) \text{ is defined}\}|}{|N_F^{\text{rel}}|}. \quad (109)$$

Similarly, let

$$R_F^{\text{rel}} \subseteq R_F \quad (110)$$

denote the governance-relevant native relationships. Relationship Projection Coverage is defined as

$$C_R(F) = \frac{|\{r \in R_F^{\text{rel}} \mid \psi_F(r) \text{ is defined}\}|}{|R_F^{\text{rel}}|}. \quad (111)$$

The two measures are reported separately because a projection may preserve all major entities while losing routing, dependency, delegation, or containment semantics between them.

An aggregate Projection Coverage score may be computed as

$$C_P(F) = \frac{|N_F^{\text{proj}}| + |R_F^{\text{proj}}|}{|N_F^{\text{rel}}| + |R_F^{\text{rel}}|}, \quad (112)$$

where

$$N_F^{\text{proj}} = \{n \in N_F^{\text{rel}} \mid \phi_F(n) \text{ is defined}\} \quad (113)$$

and

$$R_F^{\text{proj}} = \{r \in R_F^{\text{rel}} \mid \psi_F(r) \text{ is defined}\}. \quad (114)$$

The aggregate score provides a concise summary, whereas $C_N(F)$ and $C_R(F)$ retain the distinction between structural and relational expressiveness.

The denominator deliberately includes only governance-relevant constructs within the declared evaluation scope. Counting all internal framework objects would introduce implementation details that ACM does not intend to model and would make coverage scores incomparable across frameworks.

I.4.2 Projection Outcomes

Each native construct is assigned one of the following projection outcomes:

- *complete*: the construct and its governance-relevant semantics are preserved in the canonical model;
- *partial*: the construct is represented, but one or more governance-relevant attributes or relationships are not preserved;
- *unsupported*: no canonical representation exists in the current ACM model;
- *out of scope*: the construct is intentionally excluded because it carries no configuration or governance significance under the evaluation protocol.

A projected entity contributes to $C_N(F)$ only when a canonical representation is produced. However, partial projections are reported separately and are not treated as semantically equivalent to complete projections.

Let

$$N_F^{\text{complete}}, \quad N_F^{\text{partial}}, \quad N_F^{\text{unsupported}} \quad (115)$$

denote the corresponding subsets. A stricter semantic coverage measure may therefore be defined as

$$C_N^{\text{strict}}(F) = \frac{|N_F^{\text{complete}}|}{|N_F^{\text{rel}}|}. \quad (116)$$

The difference

$$C_N(F) - C_N^{\text{strict}}(F) \quad (117)$$

quantifies the proportion of native entities that are structurally projected but only partially preserved semantically.

I.4.3 Projection Validation

The resulting canonical graph is validated before it is accepted by the governance kernel.

Validation verifies:

- conformance of each projected entity to its ACI schema;
- uniqueness and stability of logical and revision identifiers;
- integrity of content digests;
- validity of relationship endpoints and cardinalities;
- preservation of source provenance;
- explicit reporting of partial and unsupported constructs.

The adapter therefore produces both a canonical Configuration Graph and a Projection Report:

$$\text{Adapter}_F(M_F) = (G_c, \mathcal{R}_F), \quad (118)$$

where $\mathcal{R}_F$ contains the projection outcome of every in-scope native entity and relationship, together with the coverage measures

$$C_N(F), \quad C_R(F), \quad C_P(F), \quad C_N^{\text{strict}}(F). \quad (119)$$

This report makes projection loss explicit rather than allowing unsupported framework semantics to disappear silently during normalization.

References